
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 5 — Classification Algorithms

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	4
1 Thuật toán phân loại (Classification Algorithms)	5
1.1 Hai loại learner trong phân loại	5
1.2 Thuật toán phân loại phổ biến	5
1.2.1 Tóm tắt từng thuật toán	6
1.3 Ứng dụng phân loại	6
1.4 Xây dựng mô hình phân loại	7
1.5 Xây dựng classifier bằng Python	7
1.6 Metric đánh giá (tổng quan)	8
2 Hồi quy logistic (Logistic Regression)	10
2.1 Các loại hồi quy logistic	11
2.2 Giả định hồi quy logistic	11
2.3 Mô hình hồi quy logistic nhị phân	12
2.4 Triển khai hồi quy logistic nhị phân bằng Python	13
2.5 Mô hình hồi quy logistic đa thức	15
2.6 Triển khai hồi quy logistic đa thức bằng Python	15
3 Thuật toán K-Nearest Neighbors (KNN)	16
3.1 KNN hoạt động như thế nào?	16
3.2 Ví dụ minh họa	17
3.3 Xây dựng mô hình KNN	18
3.4 Triển khai KNN bằng Python	18
3.5 KNN làm bộ phân loại (Classifier)	19
3.6 KNN làm bộ hồi quy (Regressor)	20
3.7 Ưu và nhược điểm của KNN	21
3.8 Ứng dụng của KNN	21
4 Thuật toán Naive Bayes	22
4.1 Các loại thuật toán Naive Bayes	23
4.2 Triển khai Naive Bayes bằng Python	23
4.3 Ưu và nhược điểm phân loại Naive Bayes	25
4.4 Ứng dụng phân loại Naive Bayes	26
5 Thuật toán Decision Tree (Cây quyết định)	26
5.1 Các loại thuật toán Decision Tree	27
5.2 Triển khai thuật toán Decision Tree	28
5.2.1 Chỉ số Gini	28
5.2.2 Tạo tách (Split Creation)	28
5.2.3 Xây dựng cây (Building a Tree)	28
5.3 Dự đoán (Prediction)	29
5.4 Giả định	29
5.5 Triển khai bằng Python	29
5.6 Ví dụ triển khai đầy đủ	31

6	Support Vector Machine (SVM)	32
6.1	Cách SVM hoạt động	32
6.2	Triển khai SVM bằng Python	33
6.3	SVM Kernels	36
6.4	Tinh chỉnh tham số SVM	38
6.5	Ưu và nhược điểm	39
7	Thuật toán Random Forest	39
7.1	Cách Random Forest hoạt động	39
7.2	Ưu điểm	40
7.3	Triển khai Python	40
7.4	Ưu và nhược điểm	41
8	Confusion Matrix (Ma trận nhầm lẫn)	42
8.1	Ví dụ thực hành	43
8.2	Metric từ confusion matrix	43
8.3	Triển khai Python	44
9	Stochastic Gradient Descent (SGD)	45
9.1	Gradient Descent	45
9.2	SGD	45
9.3	Ví dụ Python — SGDClassifier trên Iris	46
9.4	Ứng dụng	47
9.5	Ưu điểm và thách thức	47

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Classification Algorithms: https://www.tutorialspoint.com/machine_learning/machine_learning_classification_algorithms.htm.

Cách dùng

- Đọc sau phần Regression Analysis.

Dặng Tấn Quý

1 Thuật toán phân loại (Classification Algorithms)

Phân loại trong học máy

Quá trình dự đoán **lớp** hoặc **nhãn danh mục** từ các giá trị quan sát hoặc điểm dữ liệu cho trước. Đầu ra có thể là “Đen”/“Trắng”, “spam”/“không spam”, v.v.

Học có giám sát

Phân loại là kỹ thuật **học có giám sát**: thuật toán huấn luyện trên dữ liệu có nhãn để dự đoán lớp của dữ liệu mới.

Toán học

Phân loại là bài toán xấp xỉ hàm ánh xạ f từ biến đầu vào X sang biến đầu ra Y ; thuộc học có giám sát vì target được cung cấp cùng tập huấn luyện.

Ví dụ phân loại nhị phân

Phát hiện spam email: hai lớp “spam” và “không spam”. Huấn luyện classifier trên email đã gán nhãn, sau đó phân loại email chưa biết.

1.1 Hai loại learner trong phân loại

Lazy Learners

Học “lười”: lưu dữ liệu huấn luyện, chỉ phân loại khi có dữ liệu kiểm tra. Ít thời gian huấn luyện, nhiều thời gian dự đoán. Ví dụ: K-nearest neighbor (KNN), case-based reasoning.

Eager Learners

Ngược lazy: xây mô hình phân loại ngay sau khi lưu dữ liệu huấn luyện, không chờ test. Nhiều thời gian huấn luyện, ít thời gian dự đoán. Ví dụ: Decision Tree, Naive Bayes, mạng nơ-ron (ANN).

1.2 Thuật toán phân loại phổ biến

Thuật toán phân loại

Kỹ thuật học có giám sát dự đoán biến mục tiêu **phân loại** từ tập feature đầu vào. Mục tiêu: học hàm ánh xạ f giữa X và Y , thường biểu diễn bằng **ranh giới quyết định** tách các lớp trong không gian feature.

Danh sách (chi tiết ở các chương sau)

Logistic Regression; K-Nearest Neighbors (KNN); Support Vector Machine (SVM); Decision Tree; Naive Bayes; Random Forest.

1.2.1 Tóm tắt từng thuật toán**Logistic Regression**

Dùng cho phân loại **nhị phân**; mô hình hóa xác suất lớp; dự đoán lớp có xác suất cao nhất. Mô hình tuyến tính tổng quát, target Bernoulli; hàm logistic ánh xạ sang $[0, 1]$.

K-Nearest Neighbors (KNN)

Học có giám sát cho phân loại và hồi quy: tìm k điểm gần nhất, dùng láng giềng để dự đoán. k là hyperparameter. Phân loại: gán lớp **xuất hiện nhiều nhất** trong k láng giềng. Hồi quy: gán trung bình giá trị k láng giềng.

Support Vector Machine (SVM)

Thuật toán linh hoạt, mạnh cho phân loại (và hồi quy); xử lý nhiều biến liên tục và phân loại.

Decision Tree

Cây phân cấp: chia dữ liệu theo feature đến khi mỗi tập con thuần lớp hoặc cùng giá trị target; tập quy tắc để dự đoán dữ liệu mới.

Naive Bayes

Dựa trên định lý Bayes; giả định feature **độc lập** (“naive”). Xác suất mẫu thuộc lớp từ xác suất từng feature.

Random Forest

Tập hợp nhiều cây quyết định, mỗi cây huấn luyện trên tập con dữ liệu khác nhau; kết hợp dự đoán.

1.3 Ứng dụng phân loại**Ứng dụng quan trọng**

Nhận dạng giọng nói; nhận dạng chữ viết tay; nhận dạng sinh trắc học; phân loại tài liệu; phân loại ảnh; lọc spam; phát hiện gian lận; nhận diện khuôn mặt.

1.4 Xây dựng mô hình phân loại

Bước 1: Chuẩn bị dữ liệu

Thu thập và tiền xử lý: làm sạch, xử lý thiếu, mã hóa biến phân loại sang số.

Bước 2: Trích chọn feature

Trích/chọn feature liên quan (tương quan, importance, PCA, ...).

Bước 3: Chọn mô hình

Chọn thuật toán phù hợp: logistic, cây, random forest, SVM, mạng nơ-ron, ...

Bước 4: Huấn luyện

Huấn luyện trên dữ liệu có nhãn; cập nhật tham số để giảm sai số dự đoán.

Bước 5: Đánh giá

Đánh giá trên tập validation: accuracy, precision, recall, F1, AUC-ROC, ...

Bước 6: Tinh chỉnh hyperparameter

Grid search, random search, Bayesian optimization cho learning rate, regularization, số lớp ẩn, v.v.

Bước 7: Triển khai

Tích hợp production, kiểm thử dữ liệu thực, giám sát hiệu năng theo thời gian.

1.5 Xây dựng classifier bằng Python

Scikit-learn

Có thể xây classifier bằng scikit-learn theo các bước sau.

Bước 1: Import

```
import sklearn
```

Bước 2: Dataset Breast Cancer Wisconsin

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()

label_names = data['target_names']
```

```
labels = data['target']
feature_names = data['feature_names']
features = data['data']
```

```
print(label_names)
```

Output: ['malignant' 'benign']

Nhân ảnh xạ nhị phân: 0 = ác tính (malignant), 1 = lành tính (benign).

```
print(feature_names[0])    # mean radius
print(feature_names[1])    # mean texture
print(features[0])
print(features[1])
```

Bước 3: Chia train/test (60/40)

```
from sklearn.model_selection import train_test_split
train, test, train_labels, test_labels = train_test_split(
    features, labels, test_size=0.40, random_state=42)
```

Bước 4: Naive Bayes và dự đoán

```
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
model = gnb.fit(train, train_labels)
preds = gnb.predict(test)
print(preds)
```

Chuỗi 0/1 là dự đoán ác tính/lành tính trên tập kiểm tra.

Bước 5: Accuracy

```
from sklearn.metrics import accuracy_score
print(accuracy_score(test_labels, preds))
```

Output mẫu: 0.951754385965 — Gaussian Naive Bayes khoảng **95.17%** chính xác.

1.6 Metric đánh giá (tổng quan)

Chọn metric

Sau khi triển khai mô hình cần đánh giá hiệu quả; metric ảnh hưởng cách so sánh thuật toán. Chi tiết confusion matrix và công thức: chương 8; dưới đây là tóm tắt theo bài gốc chương 1.

Confusion Matrix

Bảng hai chiều Actual / Predicted với TP, TN, FP, FN — cách dễ nhất đo hiệu năng phân loại đa lớp hoặc nhị phân.

		Actual	
		1	0
Predicted	1	True Positives (TP)	False Positives (FP)
	0	False Negatives (FN)	True Negatives (TN)

True Positive (TP)

Thực tế và dự đoán đều là lớp 1 (positive).

True Negative (TN)

Thực tế và dự đoán đều là lớp 0.

False Positive (FP)

Thực tế 0, dự đoán 1.

False Negative (FN)

Thực tế 1, dự đoán 0.

Confusion matrix với sklearn

```
from sklearn.metrics import confusion_matrix
preds = gnb.predict(test)
cm = confusion_matrix(test_labels, preds)
print(cm)
```

Output mẫu:

```
[[ 73   7]
 [  4 144]]
```

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

Ví dụ: $(73 + 144)/(73 + 7 + 4 + 144) = 217/228 \approx 0,952$.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Ví dụ: $73/80 = 0,915$.

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

Ví dụ: $73/77 \approx 0,948$.

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Ví dụ: $144/151 \approx 0,954$.

Chương tiếp theo

Các thuật toán phân loại và confusion matrix được trình bày chi tiết trong các chương sau.

2 Hồi quy logistic (Logistic Regression)

Giới thiệu hồi quy logistic

Hồi quy logistic là thuật toán phân loại học có giám sát dùng để dự đoán **xác suất** của biến mục tiêu. Bản chất biến phụ thuộc (target) là **dichotomous** — chỉ có hai lớp có thể.

Biến nhị phân

Nói đơn giản, biến phụ thuộc có dạng nhị phân: dữ liệu mã hóa **1** (thành công/có) hoặc **0** (thất bại/không).

Mô hình xác suất

Về mặt toán học, mô hình hồi quy logistic dự đoán $P(Y = 1)$ như một hàm của X . Đây là một trong những thuật toán ML đơn giản nhất, áp dụng cho nhiều bài toán phân loại:

phát hiện spam, dự đoán tiểu đường, phát hiện ung thư, v.v.

2.1 Các loại hồi quy logistic

Phân loại theo số lớp

Thông thường “hồi quy logistic” nghĩa là hồi quy logistic **nhị phân** với biến mục tiêu nhị phân; tuy nhiên biến mục tiêu có thể có thêm các dạng khác. Theo số lớp của biến mục tiêu, hồi quy logistic chia thành các loại sau.

Nhị phân (Binary hoặc Binomial)

Trong phân loại này, biến phụ thuộc chỉ có hai giá trị: 1 và 0. Ví dụ: thành công/thất bại, có/không, thắng/thua.

Đa thức (Multinomial)

Biến phụ thuộc có **3 lớp trở lên**, không có thứ tự hoặc không có ý nghĩa định lượng. Ví dụ: “Loại A”, “Loại B”, “Loại C”.

Thứ bậc (Ordinal)

Biến phụ thuộc có **3 lớp trở lên có thứ tự** hoặc có ý nghĩa định lượng. Ví dụ: “kém”, “tốt”, “rất tốt”, “xuất sắc”; mỗi hạng có thể gán điểm 0, 1, 2, 3.

2.2 Giả định hồi quy logistic

Giả định trước khi triển khai

Trước khi triển khai hồi quy logistic, cần lưu ý các giả định sau.

- Với hồi quy logistic nhị phân, biến mục tiêu phải **luôn nhị phân**; kết quả mong muốn được biểu diễn bởi mức nhân tố 1.
- Mô hình **không** được có đa cộng tuyến (multi-collinearity): các biến độc lập phải độc lập với nhau.
- Chỉ đưa các biến **có ý nghĩa** vào mô hình.
- Nên chọn **cỡ mẫu lớn** cho hồi quy logistic.

2.3 Mô hình hồi quy logistic nhị phân

Dạng đơn giản nhất

Dạng đơn giản nhất là hồi quy logistic nhị phân (binomial): biến mục tiêu chỉ có hai giá trị 1 hoặc 0. Cho phép mô hình hóa quan hệ giữa nhiều biến dự báo và biến mục tiêu nhị phân. Trong hồi quy logistic, hàm tuyến tính được dùng làm đầu vào cho hàm khác:

$$h_{\theta}(x) = g(\theta^T x)$$

ở đây g là hàm **logistic** hoặc **sigmoid**.

Hàm sigmoid

$$g(z) = \frac{1}{1 + e^{-z}}$$

Đường cong sigmoid có giá trị trục y nằm trong khoảng $(0, 1)$ và cắt trục tại 0,5. (Không có hình minh họa trong bộ tài liệu này; có thể vẽ $g(z)$ với $z \in [-6, 6]$ để thấy dạng chữ S.)

Phân lớp dương/âm

Các lớp có thể chia thành dương hoặc âm. Đầu ra của hàm giả thuyết được hiểu là xác suất lớp dương nếu nằm trong $(0, 1)$. Trong triển khai bài này: đầu ra $\geq 0,5 \Rightarrow$ dương; ngược lại $\Rightarrow$ âm.

Hàm mất mát $J(\theta)$

Cần định nghĩa hàm loss đo mức độ tốt của thuật toán với trọng số θ :

$$J(\theta) = \frac{1}{m} (-y^T \log(h) - (1 - y)^T \log(1 - h))$$

Mục tiêu chính: **tối thiểu hóa** hàm loss — điều chỉnh trọng số (tăng/giảm). Nhờ đạo hàm của loss theo từng trọng số, biết tham số nào nên lớn hoặc nhỏ hơn.

Gradient descent

Phương trình gradient descent cho biết loss thay đổi thế nào khi sửa tham số:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} X^T (h - y)$$

2.4 Triển khai hồi quy logistic nhị phân bằng Python

Dataset Iris (hai feature đầu)

Dùng bộ hoa iris: 3 lớp, mỗi lớp 50 mẫu; chỉ lấy **hai cột feature đầu**. Mỗi lớp là một loài iris; chuyển bài toán thành phân loại nhị phân (lớp 0 vs các lớp khác).

Import và nạp dữ liệu

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets

iris = datasets.load_iris()
X = iris.data[:, :2]
y = (iris.target != 0) * 1
```

Vẽ dữ liệu huấn luyện

```
plt.figure(figsize=(6, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1],
            color='g', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1],
            color='y', label='1')
plt.legend()
plt.show()
```

Lớp LogisticRegression thủ công

Định nghĩa sigmoid, loss và gradient descent:

```
class LogisticRegression:
    def __init__(self, lr=0.01, num_iter=100000,
                 fit_intercept=True, verbose=False):
        self.lr = lr
        self.num_iter = num_iter
        self.fit_intercept = fit_intercept
        self.verbose = verbose

    def __add_intercept(self, X):
        intercept = np.ones((X.shape[0], 1))
        return np.concatenate((intercept, X), axis=1)

    def __sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def __loss(self, h, y):
        return (-y * np.log(h) - (1 - y) * np.log(1 - h)).mean()
```

```

def fit(self, X, y):
    if self.fit_intercept:
        X = self.__add_intercept(X)
    self.theta = np.zeros(X.shape[1])
    for i in range(self.num_iter):
        z = np.dot(X, self.theta)
        h = self.__sigmoid(z)
        gradient = np.dot(X.T, (h - y)) / y.size
        self.theta -= self.lr * gradient
        z = np.dot(X, self.theta)
        h = self.__sigmoid(z)
        loss = self.__loss(h, y)
        if self.verbose and i % 10000 == 0:
            print(f'loss: {loss} \t')

def predict_prob(self, X):
    if self.fit_intercept:
        X = self.__add_intercept(X)
    return self.__sigmoid(np.dot(X, self.theta))

def predict(self, X):
    return self.predict_prob(X).round()

```

Huấn luyện, dự đoán và đánh giá

```

model = LogisticRegression(lr=0.1, num_iter=300000)
preds = model.predict(X)
(preds == y).mean()

```

Vẽ biên quyết định (đường contour xác suất 0.5):

```

plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1],
            color='g', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1],
            color='y', label='1')
plt.legend()
x1_min, x1_max = X[:, 0].min(), X[:, 0].max()
x2_min, x2_max = X[:, 1].min(), X[:, 1].max()
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max),
                        np.linspace(x2_min, x2_max))
grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model.predict_prob(grid).reshape(xx1.shape)
plt.contour(xx1, xx2, probs, [0.5], linewidths=1, colors='red')
plt.show()

```

Đánh giá mô hình nhị phân

Tỷ lệ `(preds == y).mean()` là độ chính xác trên toàn bộ tập đã dùng để fit (bài gốc). Trong thực hành nên tách train/test để đánh giá không chệch.

2.5 Mô hình hồi quy logistic đa thức

Multinomial logistic regression

Dạng hữu ích khác: hồi quy logistic **đa thức** — biến mục tiêu có 3 lớp trở lên **không có thứ tự**, không có ý nghĩa định lượng.

2.6 Triển khai hồi quy logistic đa thức bằng Python

Dataset digits

Dùng tập digits từ scikit-learn (chữ số viết tay 0–9).

Import thư viện

```
from sklearn import datasets
from sklearn import linear_model
from sklearn import metrics
from sklearn.model_selection import train_test_split
```

Nạp digits và chia tập

```
digits = datasets.load_digits()
X = digits.data
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=1)
```

Huấn luyện và đánh giá

```
digreg = linear_model.LogisticRegression()
digreg.fit(X_train, y_train)
y_pred = digreg.predict(X_test)
print("Accuracy of Logistic Regression model is:",
      metrics.accuracy_score(y_test, y_pred) * 100)
```

Output:

Accuracy of Logistic Regression model is: 95.6884561891516

Kết quả độ chính xác

Độ chính xác mô hình khoảng **96%** (cụ thể $\approx 95,69\%$ trên tập test với `test_size=0.4`, `random_state=1`).

3 Thuật toán K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN)

K-nearest neighbors (KNN) là thuật toán học có giám sát dùng cho cả **phân loại** và **hồi quy**. Trong công nghiệp, KNN chủ yếu dùng cho bài toán phân loại. Ý tưởng: tìm k điểm dữ liệu gần nhất với điểm kiểm tra và dùng các láng giềng đó để dự đoán. k là siêu tham số cần tinh chỉnh — số láng giềng xét đến.

Phân loại

Với phân loại, KNN gán điểm kiểm tra vào **lớp xuất hiện nhiều nhất** trong k láng giềng gần nhất (bỏ phiếu đa số).

Hồi quy

Với hồi quy, KNN gán điểm kiểm tra bằng **trung bình** giá trị của k láng giềng gần nhất.

Metric khoảng cách

Metric đo độ tương tự giữa hai điểm ảnh hưởng mạnh đến hiệu năng KNN. Phổ biến: khoảng cách **Euclidean**, **Manhattan**, **Minkowski**.

Hai đặc tính định nghĩa KNN

- **Lazy learning**: không có giai đoạn huấn luyện chuyên biệt; dùng toàn bộ dữ liệu khi phân loại.
- **Non-parametric**: không giả định gì về phân phối dữ liệu nền.

3.1 KNN hoạt động như thế nào?

Feature similarity

KNN dùng “độ tương đồng feature” để dự đoán giá trị điểm mới: điểm mới được gán nhãn dựa trên mức độ khớp với tập huấn luyện.

Bước 1 — Dataset

Để triển khai bất kỳ thuật toán nào cần có dataset. Ở bước đầu của KNN: nạp dữ liệu **huấn luyện** và **kiểm tra**.

Bước 2 — Chọn K

Chọn giá trị K (số điểm láng giềng gần nhất). K có thể là bất kỳ số nguyên nào.

Bước 3 — Với mỗi điểm trong tập test

3.1 Tính khoảng cách giữa điểm test và mỗi hàng dữ liệu huấn luyện (Euclidean, Manhattan hoặc Hamming; phổ biến nhất là Euclidean).

3.2 Sắp xếp theo khoảng cách tăng dần.

3.3 Chọn K hàng đầu từ mảng đã sắp xếp.

3.4 Gán lớp cho điểm test theo **lớp xuất hiện nhiều nhất** trong K hàng đó.

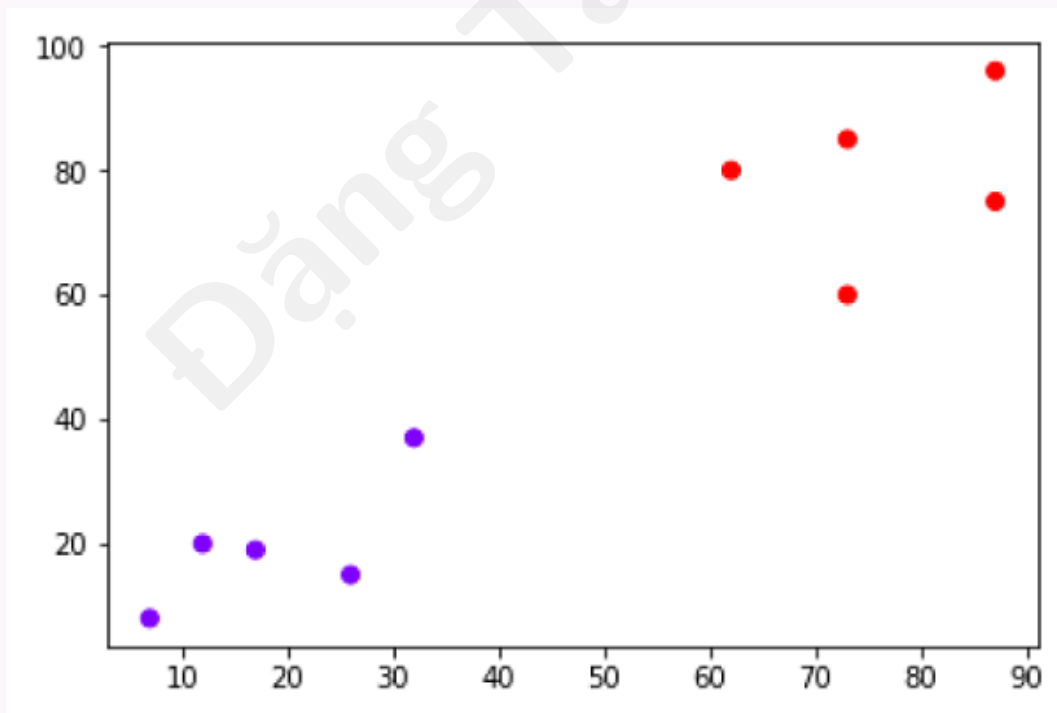
Bước 4 — Kết thúc

Kết thúc quy trình cho mọi điểm test.

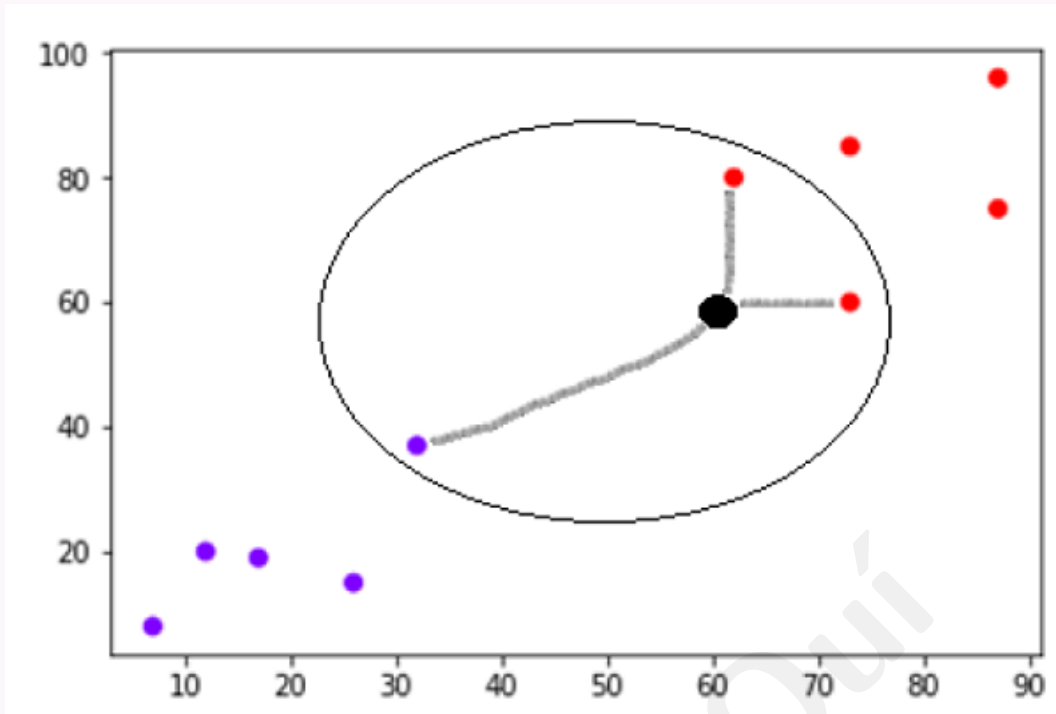
3.2 Ví dụ minh họa

Phân loại điểm mới với $K = 3$

Giả sử có dataset có thể vẽ như sau (hai lớp màu):



Cần phân loại điểm mới (chấm đen tại $(60, 60)$) vào lớp xanh hoặc đỏ. Giả sử $K = 3$: tìm ba điểm gần nhất.



Trong ba láng giềng, hai thuộc lớp Đỏ $\Rightarrow$ chấm đen được gán lớp Đỏ.

3.3 Xây dựng mô hình KNN

Các bước xây mô hình

1. **Nạp dữ liệu:** dùng pandas, numpy, hoặc `sklearn.datasets`.
2. **Chia dữ liệu:** tập huấn luyện và kiểm tra.
3. **Chuẩn hóa:** đảm bảo mỗi feature đóng góp như nhau vào khoảng cách.
4. **Tính khoảng cách:** giữa điểm test và từng điểm huấn luyện.
5. **Chọn k láng giềng:** theo khoảng cách đã tính.
6. **Dự đoán:** bỏ phiếu đa số (phân loại) hoặc trung bình (hồi quy).
7. **Đánh giá:** accuracy, precision, recall, F1-score, v.v.

3.4 Triển khai KNN bằng Python

Classifier và Regressor

KNN dùng được cho cả phân loại và hồi quy; dưới đây là hai “recipe” trong Python.

3.5 KNN làm bộ phân loại (Classifier)

Import và nạp Iris

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn import datasets

iris = datasets.load_iris()
headers = ['sepal-length', 'sepal-width',
           'petal-length', 'petal-width', 'Class']
dataset = pd.DataFrame(
    np.column_stack([iris.data, iris.target_names[iris.target]]),
    columns=headers)
dataset.head()
```

Output mẫu head():

	sepal-length	sepal-width	petal-length	petal-width	Class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Tiền xử lý và chia tập

```
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 4].values

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.40)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Huấn luyện và dự đoán

```
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=8)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

Báo cáo kết quả

```

from sklearn.metrics import (classification_report,
                             confusion_matrix, accuracy_score)

result = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(result)
result1 = classification_report(y_test, y_pred)
print("Classification Report:")
print(result1)
result2 = accuracy_score(y_test, y_pred)
print("Accuracy:", result2)

```

Output:

Confusion Matrix:

```

[[21  0  0]
 [ 0 16  0]
 [ 0  7 16]]

```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	21
Iris-versicolor	0.70	1.00	0.82	16
Iris-virginica	1.00	0.70	0.82	23
micro avg	0.88	0.88	0.88	60
macro avg	0.90	0.90	0.88	60
weighted avg	0.92	0.88	0.88	60

Accuracy: 0.8833333333333333

3.6 KNN làm bộ hồi quy (Regressor)**Dữ liệu hai feature, target petal-length**

```

import numpy as np
import pandas as pd
from sklearn import datasets

iris = datasets.load_iris()
headernames = ['sepal-length', 'sepal-width',
               'petal-length', 'petal-width', 'Class']
data = pd.DataFrame(
    np.column_stack([iris.data, iris.target_names[iris.target]]),
    columns=headernames)
array = data.values
X = array[:, :2]
Y = array[:, 2]
data.shape

```

Output: (150, 5)

KNeighborsRegressor và MSE

```
from sklearn.neighbors import KNeighborsRegressor

knnr = KNeighborsRegressor(n_neighbors=10)
knnr.fit(X, Y)
print("The MSE is:",
      format(np.power(Y - knnr.predict(X), 2).mean()))
```

Output:

The MSE is: 0.12226666666666669

3.7 Ưu và nhược điểm của KNN

Ưu điểm (Pros)

- Thuật toán rất **đơn giản**, dễ hiểu và diễn giải.
- Hữu ích với dữ liệu **phi tuyến** vì không giả định dạng phân phối.
- **Đa năng**: dùng cho phân loại và hồi quy.
- Độ chính xác tương đối cao, dù còn nhiều mô hình có giám sát tốt hơn KNN.

Nhược điểm (Cons)

- **Tốn tính toán**: lưu toàn bộ dữ liệu huấn luyện.
- Cần **bộ nhớ lớn** so với thuật toán có giám sát khác.
- Dự đoán **chậm** khi N lớn.
- **Nhạy thang đo** dữ liệu và feature không liên quan.

3.8 Ứng dụng của KNN

Ngân hàng

KNN dự đoán cá nhân có phù hợp để duyệt vay không: có đặc điểm giống nhóm vỡ nợ không?

Xếp hạng tín dụng

So sánh với người có đặc điểm tương tự để ước lượng điểm tín dụng.

Chính trị

Phân loại cử tri tiềm năng: “Sẽ bầu”, “Không bầu”, “Bầu đảng X”, “Bầu đảng Y”, v.v.

Lĩnh vực khác

Nhận dạng giọng nói, chữ viết tay, ảnh và video.

4 Thuật toán Naive Bayes

Naive Bayes là gì?

Thuật toán Naive Bayes là thuật toán phân loại dựa trên **định lý Bayes**. Thuật toán giả định các feature **độc lập** với nhau — vì vậy gọi là “naive” (ngây thơ). Tính xác suất mẫu thuộc một lớp dựa trên xác suất của từng feature.

Ví dụ điện thoại thông minh

Điện thoại có thể được coi là “thông minh” nếu có màn hình cảm ứng, internet, camera tốt, v.v. Dù các feature phụ thuộc nhau trong thực tế, mỗi feature vẫn **đóng góp độc lập** vào xác suất điện thoại thông minh.

Xác suất hậu nghiệm trong phân loại Bayes

Trong phân loại Bayes, quan tâm chính là xác suất hậu nghiệm: xác suất nhận L khi đã quan sát feature, $P(L | \text{features})$. Theo định lý Bayes:

$$P(L | \text{features}) = \frac{P(L) P(\text{features} | L)}{P(\text{features})}$$

Ý nghĩa các thành phần

- $P(L | \text{features})$: xác suất hậu nghiệm của lớp;
- $P(L)$: xác suất tiên nghiệm (prior) của lớp;
- $P(\text{features} | L)$: likelihood — xác suất predictor khi biết lớp;
- $P(\text{features})$: xác suất tiên nghiệm của predictor.

Quy trình Naive Bayes

Dùng định lý Bayes tính xác suất mẫu thuộc từng lớp. Với mỗi feature, tính $P(\text{feature} | L)$ rồi **nhân** các likelihood (giả định độc lập). Nhân likelihood với prior $P(L)$ để có posterior. Lặp cho mọi lớp; chọn lớp có xác suất cao nhất.

4.1 Các loại thuật toán Naive Bayes

Nhiều biến thể

Có nhiều loại Naive Bayes; phần này trình bày ba loại phổ biến.

Gaussian Naive Bayes

Gaussian Naive Bayes là bộ phân loại Naive Bayes đơn giản nhất: giả định dữ liệu mỗi nhãn được rút từ phân phối Gaussian. Dùng khi feature là biến **liên tục** tuân theo phân phối chuẩn.

Multinomial Naive Bayes

Giả định feature được rút từ phân phối **Multinomial**. Phù hợp feature dạng **đếm rời rạc**; thường dùng phân loại văn bản (tần suất từ trong tài liệu).

Bernoulli Naive Bayes

Giả định feature **nhị phân** (0 và 1). Phân loại văn bản với mô hình “bag of words” có thể dùng Bernoulli Naive Bayes.

4.2 Triển khai Naive Bayes bằng Python

Gaussian Naive Bayes

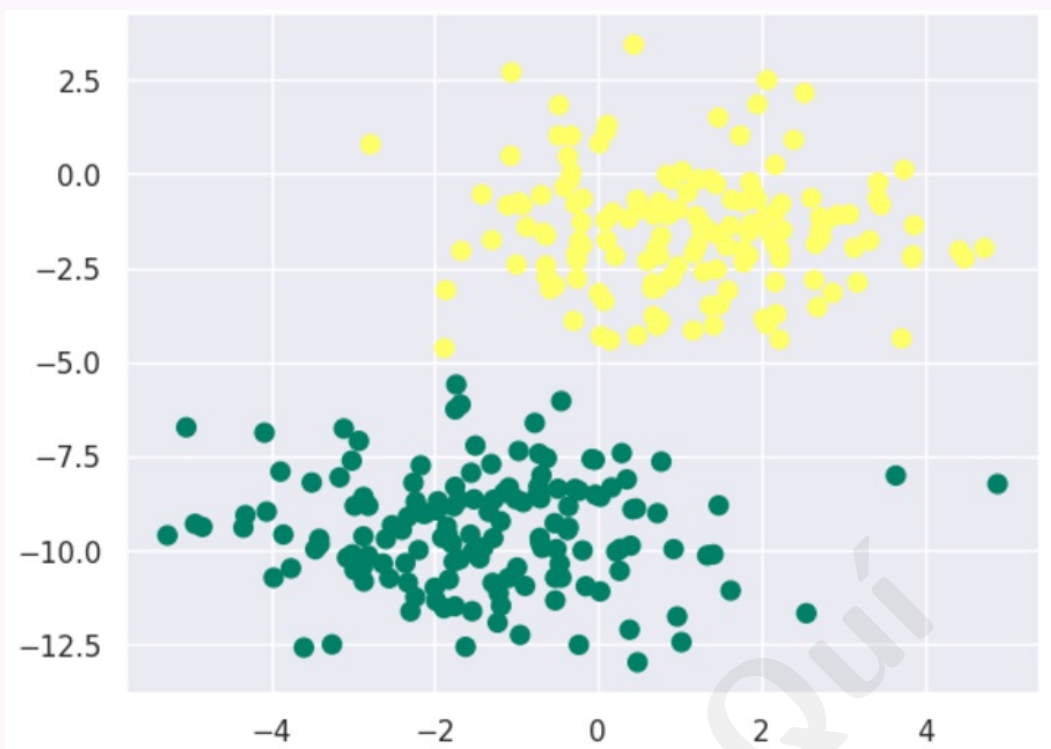
Tùy dataset chọn mô hình phù hợp; ở đây triển khai **Gaussian Naive Bayes** bằng scikit-learn.

Import

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

Tạo blob phân phối Gaussian

```
from sklearn.datasets import make_blobs
X, y = make_blobs(300, 2, centers=2, random_state=2,
                  cluster_std=1.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='summer')
plt.show()
```



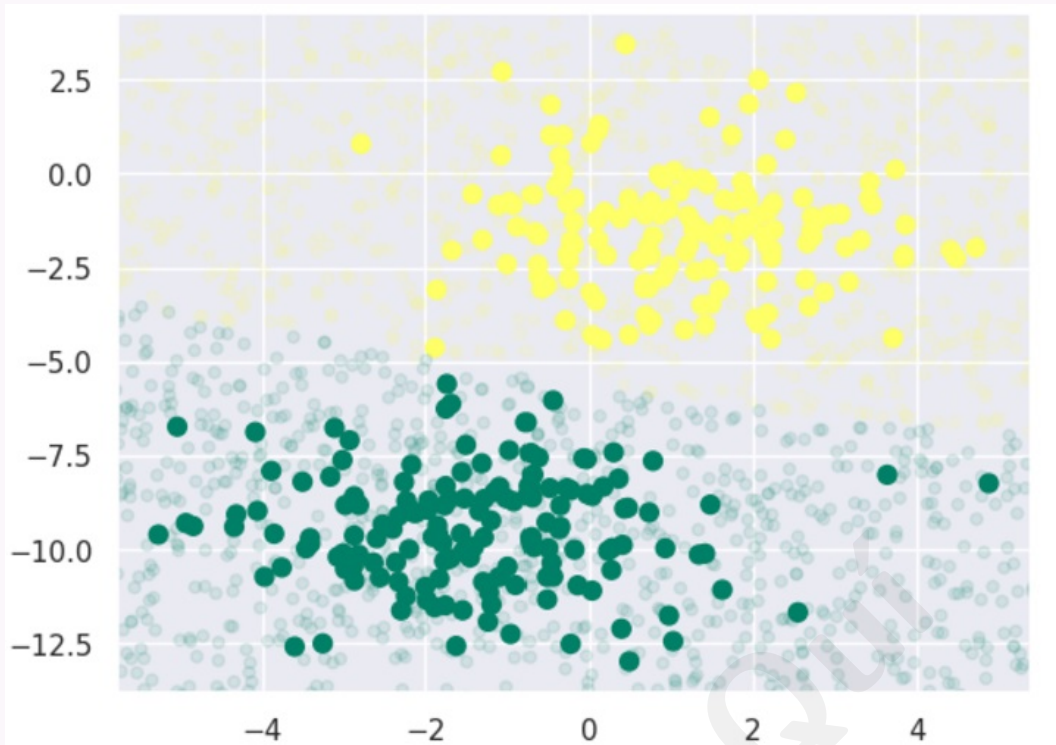
Huấn luyện GaussianNB

```
from sklearn.naive_bayes import GaussianNB
model_GNB = GaussianNB()
model_GNB.fit(X, y)
```

Dự đoán trên dữ liệu mới

```
rng = np.random.RandomState(0)
Xnew = [-6, -14] + [14, 18] * rng.rand(2000, 2)
ynew = model_GNB.predict(Xnew)

plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='summer')
lim = plt.axis()
plt.scatter(Xnew[:, 0], Xnew[:, 1], c=ynew, s=20,
            cmap='summer', alpha=0.1)
plt.axis(lim)
plt.show()
```

Xác suất hậu nghiệm

```
yprob = model_GNB.predict_proba(Xnew)
yprob[-10:].round(3)
```

Output:

```
array([[0.998, 0.002],
       [1.    , 0.    ],
       [0.987, 0.013],
       [1.    , 0.    ],
       [1.    , 0.    ],
       [1.    , 0.    ],
       [1.    , 0.    ],
       [1.    , 0.    ],
       [0.    , 1.    ],
       [0.986, 0.014]])
```

4.3 Ưu và nhược điểm phân loại Naive Bayes

Ưu điểm (Pros)

- Dễ triển khai và **nhANH**.
- Hội tụ nhanh hơn mô hình phân biệt như logistic regression.
- Cần ít dữ liệu huấn luyện.
- Mở rộng tuyến tính theo số predictor và số điểm dữ liệu.

- Dự đoán xác suất; xử lý dữ liệu liên tục và rời rạc.
- Dùng cho phân loại **nhị phân** và **đa lớp**.

Nhược điểm (Cons)

- Giả định **độc lập feature** mạnh — trong thực tế gần như không bao giờ đúng tuyệt đối.
- Vấn đề **zero frequency**: nếu một giá trị phân loại chưa xuất hiện trong tập huấn luyện, mô hình gán xác suất 0 và không dự đoán được (cần làm trơn Laplace trong thực hành).

4.4 Ứng dụng phân loại Naive Bayes

Dự đoán thời gian thực

Triển khai và tính toán nhanh $\Rightarrow$ phù hợp dự đoán real-time.

Dự đoán đa lớp

Ước lượng xác suất hậu nghiệm của nhiều lớp biến mục tiêu.

Phân loại văn bản

Phù hợp spam-filtering, phân tích cảm xúc (sentiment analysis).

Hệ gợi ý (Recommendation system)

Kết hợp collaborative filtering: lọc thông tin chưa thấy, dự đoán người dùng có thích tài nguyên hay không.

5 Thuật toán Decision Tree (Cây quyết định)

Decision Tree Algorithm

Thuật toán cây quyết định là thuật toán phân cấp dạng **cây** dùng để phân loại hoặc dự đoán kết quả theo tập quy tắc. Chia dữ liệu thành các tập con theo giá trị feature đầu vào; lặp đệ quy cho đến khi mỗi tập con thuộc cùng một lớp hoặc cùng giá trị biến mục tiêu. Cây thu được là tập quy tắc quyết định để dự đoán hoặc phân loại dữ liệu mới.

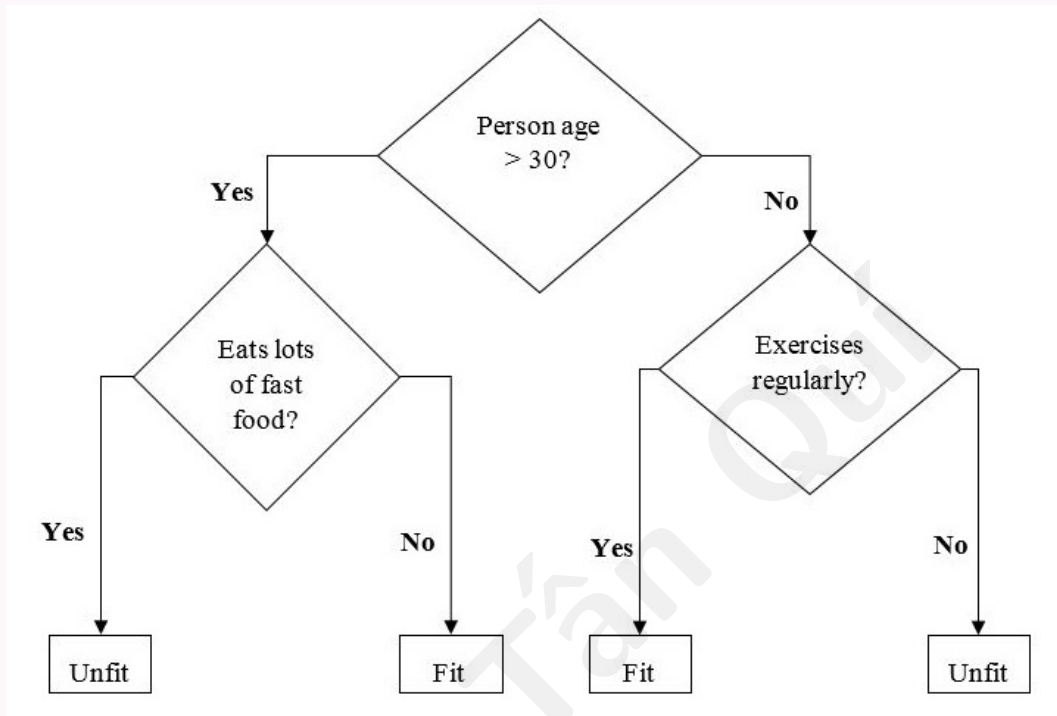
Chọn feature tách tại mỗi nút

Tại mỗi nút, chọn feature **tốt nhất** để tách dữ liệu — feature mang lại nhiều **information gain** nhất hoặc giảm **entropy** nhiều nhất. Information gain đo lượng thông tin thu được khi tách tại một feature; entropy đo độ ngẫu nhiên/hỗn loạn trong dữ liệu. Thuật toán

dùng các đại lượng này để quyết định feature tách tại mỗi nút.

Ví dụ cây nhị phân

Ví dụ cây nhị phân dự đoán người “fit” hay “unfit” theo tuổi, thói quen ăn uống và tập luyện:



Trong cây trên, các **câu hỏi** là nút quyết định (decision nodes); **kết quả cuối** là lá (leaves).

5.1 Các loại thuật toán Decision Tree

Classification Tree

Cây phân loại dùng để xếp dữ liệu vào các **lớp** hoặc **hạng mục** khác nhau. Tách tập con theo feature và gán mỗi tập con một lớp.

Regression Tree

Cây hồi quy dùng để dự đoán giá trị **số** (biến liên tục). Tách tập con theo feature và gán mỗi tập con một giá trị số.

5.2 Triển khai thuật toán Decision Tree

5.2.1 Chỉ số Gini

Gini Index

Gini Index là tên hàm **cost** đánh giá các tách nhị phân trong dataset, làm việc với biến mục tiêu phân loại Success/Failure (hoặc hai lớp). Gini càng **cao** thì tập con càng **đồng nhất** (homogeneity). Gini hoàn hảo = 0; tệ nhất = 0.5 (bài toán 2 lớp).

Tính Gini cho một tách

1. Tính Gini cho từng nút con bằng công thức $p^2 + q^2$ (tổng bình phương xác suất thành công và thất bại).
2. Tính Gini cho tách bằng **Gini có trọng số** của từng nút con.

Thuật toán CART (Classification and Regression Tree) dùng phương pháp Gini để tạo tách nhị phân.

5.2.2 Tạo tách (Split Creation)

Split

Một tách gồm một **thuộc tính** trong dataset và một **giá trị**. Tách dataset thành hai danh sách hàng (nhóm trái/phải) theo chỉ số thuộc tính và giá trị tách. Sau khi có hai nhóm, tính giá trị tách bằng Gini ở bước trên; giá trị tách quyết định thuộc tính nằm ở nhóm nào.

Đánh giá mọi tách

Kiểm tra mọi giá trị gắn với mỗi thuộc tính như ứng viên tách. Tìm tách tốt nhất bằng cách đánh giá **chi phí** tách; tách tốt nhất trở thành nút trong cây.

5.2.3 Xây dựng cây (Building a Tree)

Nút gốc và nút lá

Cây có nút gốc (root) và nút lá (terminal nodes). Sau khi tạo nút gốc, xây cây theo hai phần sau.

Phần 1: Tạo nút lá (terminal node)

Quyết định khi nào **dừng** mở rộng cây, dùng hai tiêu chí:

- **Maximum Tree Depth:** số nút tối đa sau nút gốc; dừng thêm nút lá khi đạt độ sâu tối đa.
- **Minimum Node Records:** số mẫu huấn luyện tối thiểu một nút chịu trách nhiệm; dừng khi đạt hoặc dưới ngưỡng này.

Nút lá dùng để đưa ra **dự đoán cuối cùng**.

Phần 2: Tách đệ quy (Recursive Splitting)

Sau khi tạo một nút, tạo nút con **đệ quy** trên từng nhóm dữ liệu sinh ra bởi tách — gọi lại cùng hàm nhiều lần.

5.3 Dự đoán (Prediction)

Điều hướng trên cây

Sau khi xây cây, dự đoán bằng cách **điều hướng** cây với một hàng dữ liệu cụ thể. Dùng hàm đệ quy tương tự khi xây cây: gọi lại routine dự đoán với nút con trái hoặc phải.

5.4 Giả định

Giả định khi tạo decision tree

- Tập huấn luyện là **nút gốc**.
- Bộ phân loại cây quyết định **ưu tiên** feature dạng phân loại (categorical). Nếu dùng giá trị liên tục cần **rời rạc hóa** (discretize) trước khi xây mô hình.
- Bản ghi được phân phối **đệ quy** theo giá trị thuộc tính.
- Dùng **phương pháp thống kê** để đặt thuộc tính ở vị trí nút (gốc hoặc nút trong).

5.5 Triển khai bằng Python

Dataset Iris

150 mẫu hoa iris, bốn feature: chiều dài/rộng đài, chiều dài/rộng cánh hoa. Ba lớp: setosa, versicolor, virginica.

Import, nạp dữ liệu và chia tập

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=0)
```

Huấn luyện và dự đoán

```
dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
y_pred = dtc.predict(X_test)
```

Độ chính xác

```
accuracy = np.sum(y_pred == y_test) / len(y_test)
print("Accuracy:", accuracy)
```

Output:

Accuracy: 0.9777777777777777

Trực quan hóa cây

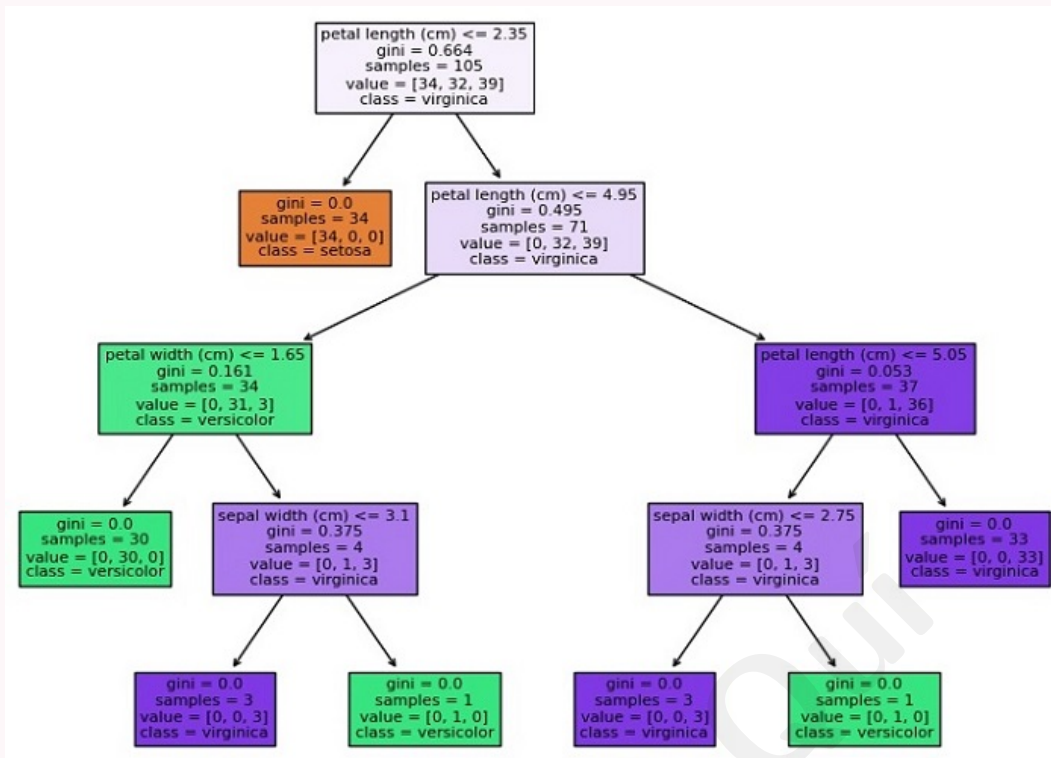
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))
plot_tree(dtc, filled=True,
          feature_names=iris.feature_names,
          class_names=iris.target_names)
plt.show()
```

Hàm `plot_tree` vẽ cấu trúc cây: mỗi nút là quyết định theo feature; mỗi lá là lớp hoặc giá trị số. Tham số `filled=True` tô màu nút; `feature_names` và `class_names` gắn nhãn.

Biểu đồ cây quyết định

Biểu đồ minh họa:



Màu nút thể hiện lớp/ giá trị chiếm đa số trong nút; số ở đáy là số mẫu đi tới nút đó.

5.6 Ví dụ triển khai đầy đủ

Mã hoàn chỉnh

```

import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=0)

dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
y_pred = dtc.predict(X_test)

accuracy = np.sum(y_pred == y_test) / len(y_test)
print("Accuracy:", accuracy)

plt.figure(figsize=(20, 10))
plot_tree(dtc, filled=True,
          feature_names=iris.feature_names,
  
```

```
class_names=iris.target_names)
plt.show()
```

Output: Accuracy: 0.9777777777777777 (khoảng **97,8%**).

6 Support Vector Machine (SVM)

Support Vector Machine (SVM)

Thuật toán học có giám sát mạnh và linh hoạt, dùng cho phân loại và hồi quy; thường dùng cho **phân loại**. Ra đời từ thập niên 1960, hoàn thiện thập niên 1990; phổ biến nhờ xử lý nhiều biến liên tục và phân loại.

6.1 Cách SVM hoạt động

Mục tiêu

Tìm **siêu phẳng** (hyperplane) tách các lớp; tối đa hóa **margin** — khoảng cách từ siêu phẳng đến các điểm gần nhất mỗi lớp (support vectors).

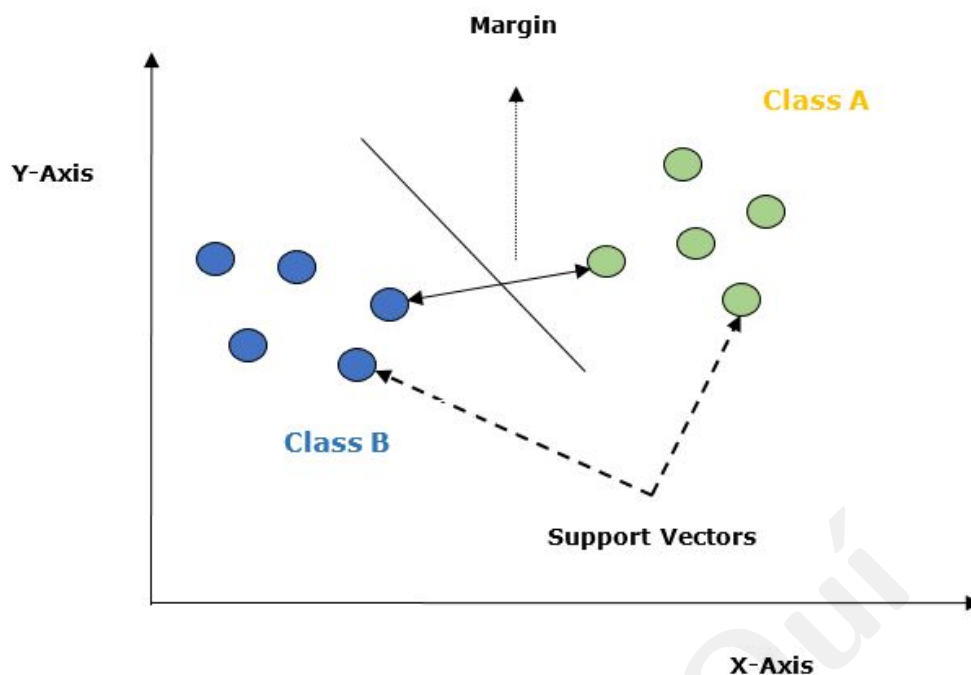
Khoảng cách đến siêu phẳng

$$\text{distance} = \frac{w \cdot x + b}{\|w\|}$$

w : vector trọng số (vuông góc siêu phẳng); b : bias; $\|w\|$: chuẩn Euclid của w .

Bài toán tối ưu

Tối đa hóa margin với ràng buộc mọi điểm được phân loại đúng — bài toán lồi, giải bằng quadratic programming. Nếu không tách tuyến tính được: dùng **kernel trick** ánh xạ lên không gian chiều cao hơn mà không tính ánh xạ tường minh.



Support Vectors

Điểm dữ liệu gần siêu phẳng nhất; định nghĩa đường phân tách.

Hyperplane

Mặt phẳng quyết định chia không gian thành vùng lớp khác nhau (đường thẳng trong 2D).

Margin

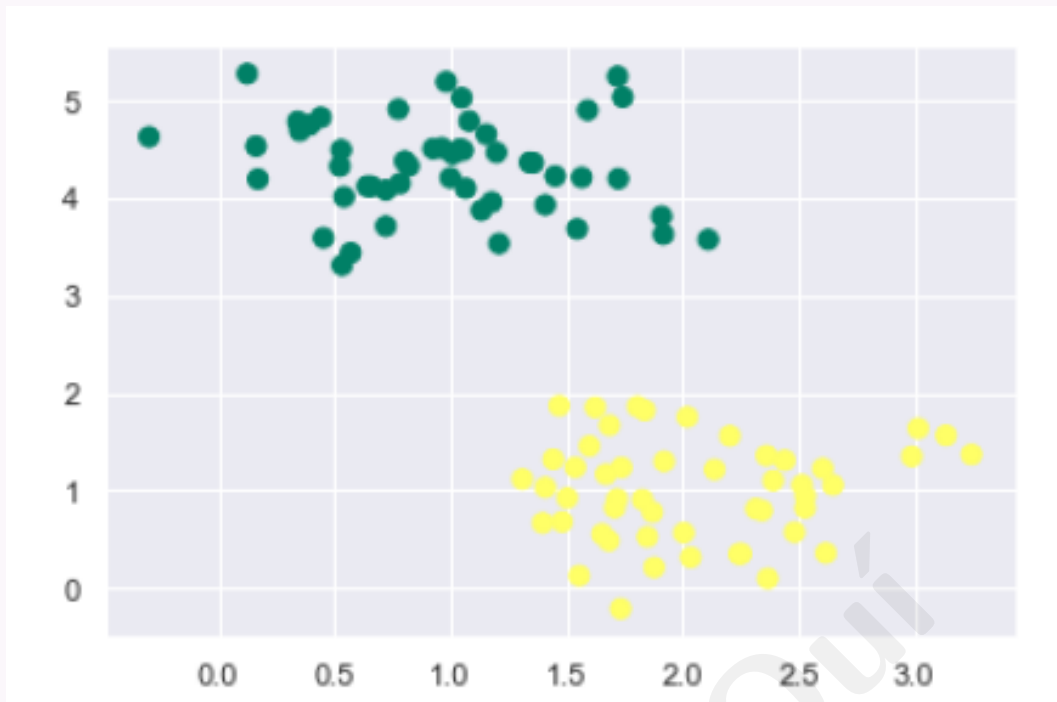
Khoảng cách giữa hai đường song song qua support vectors gần nhất hai lớp; margin lớn thường tốt hơn margin nhỏ.

6.2 Triển khai SVM bằng Python

Import và dữ liệu mẫu

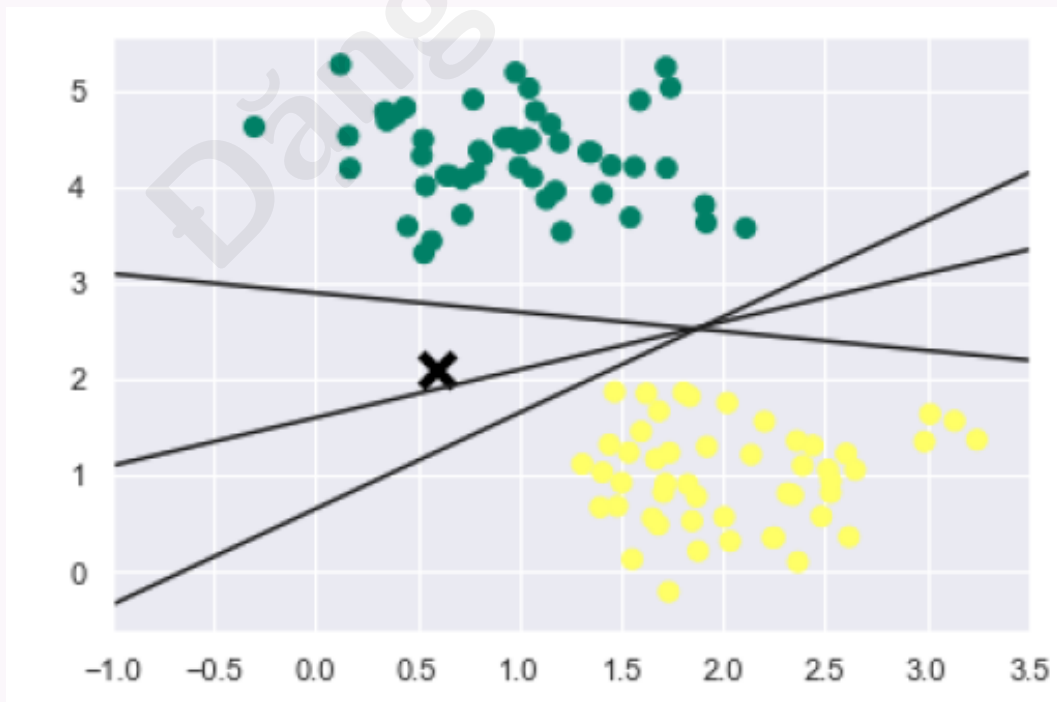
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=100, centers=2, random_state=0,
                  cluster_std=0.50)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='summer')
```



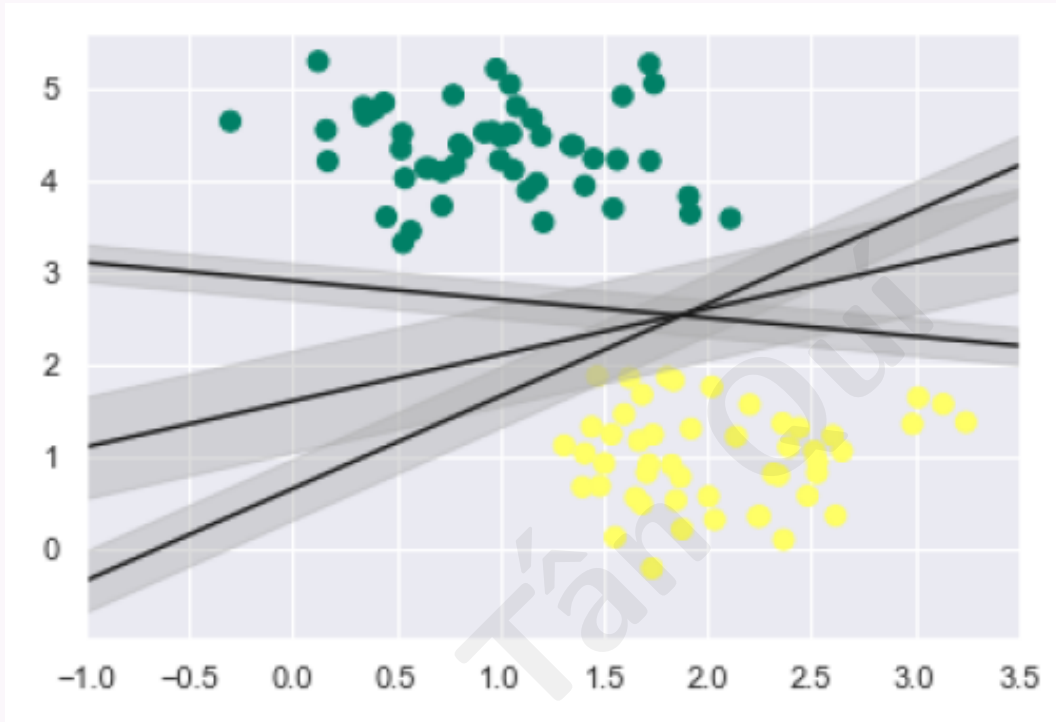
Nhiều đường phân tách có thể

```
xfit = np.linspace(-1, 3.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='summer')
for m, b in [(1, 0.65), (0.5, 1.6), (-0.2, 2.9)]:
    plt.plot(xfit, m * xfit + b, '-k')
plt.xlim(-1, 3.5)
```



Maximum Marginal Hyperplane (MMH)

```
for m, b, d in [(1, 0.65, 0.33), (0.5, 1.6, 0.55), (-0.2, 2.9, 0.2)]:
    yfit = m * xfit + b
    plt.plot(xfit, yfit, '-k')
    plt.fill_between(xfit, yfit - d, yfit + d, color='#AAAAAA', alpha=0.4)
plt.xlim(-1, 3.5)
```



SVM chọn đường tối đa hóa margin.

SVC kernel tuyến tính

```
from sklearn.svm import SVC
model = SVC(kernel='linear', C=1E10)
model.fit(X, y)
```

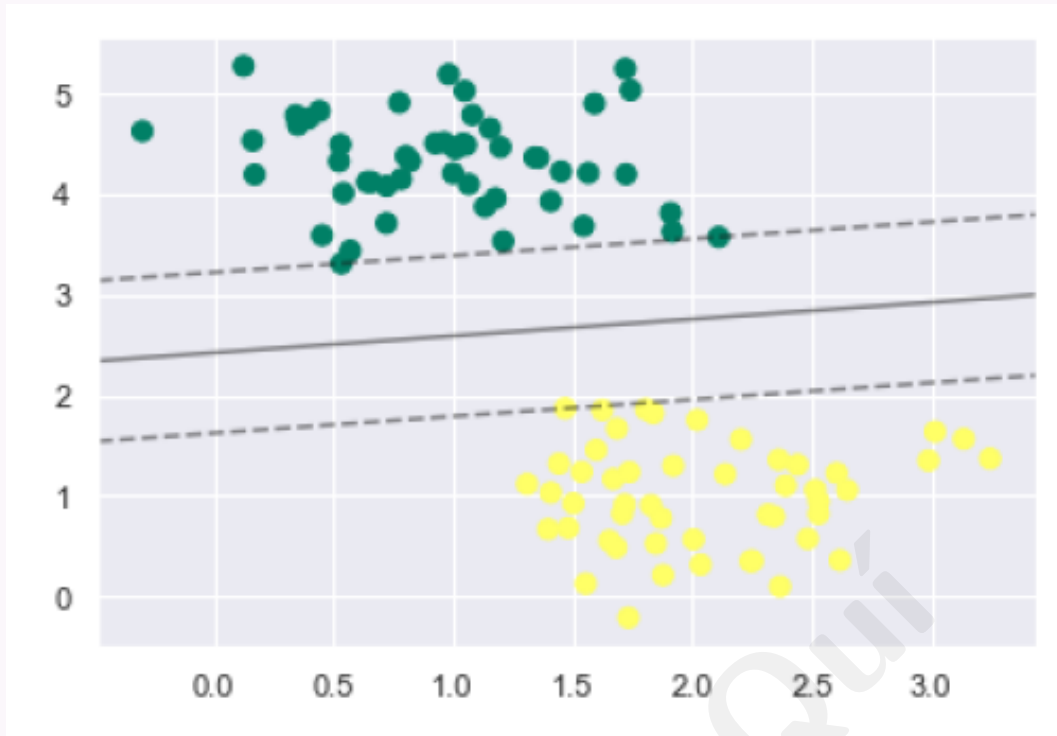
Hàm decision_function và support vectors

Vẽ biên quyết định và support vectors (xem code đầy đủ trong notebook); sau khi fit:

```
print(model.support_vectors_)
```

Output mẫu:

```
[[0.5323772  3.31338909]
 [2.11114739 3.57660449]
 [1.46870582 1.86947425]]
```



6.3 SVM Kernels

Kernel trick

Kernel biến không gian đầu vào chiều thấp sang dạng phù hợp; biến bài toán không tách tuyến tính thành tách được trong không gian mới.

Linear kernel

$$k(x, x_i) = \sum x \cdot x_i \text{ (tích vô hướng).}$$

Polynomial kernel

$$K(x, x_i) = 1 + \sum (x \cdot x_i)^d; d \text{ là bậc đa thức (chỉ định thủ công).}$$

RBF kernel

$$K(x, x_i) = \exp(-\gamma \sum (x - x_i)^2); \gamma \in [0, 1], \text{ mặc định thường } 0.1.$$

SVM với kernel trên Iris (2 feature)

```
from sklearn import svm, datasets
import matplotlib.pyplot as plt
import numpy as np

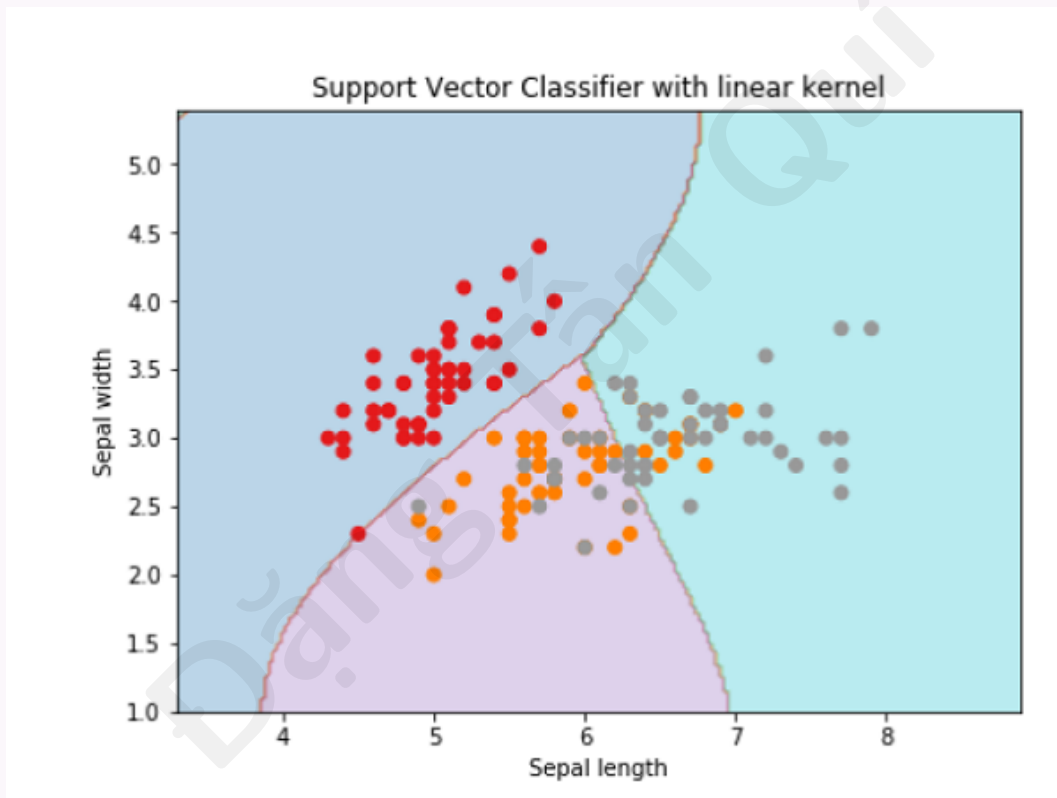
iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target
```

```

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
h = (x_max - x_min) / 100
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
X_plot = np.c_[xx.ravel(), yy.ravel()]
C = 1.0

svc = svm.SVC(kernel='linear', C=C).fit(X, y)
Z = svc.predict(X_plot).reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title('SVC with linear kernel')

```

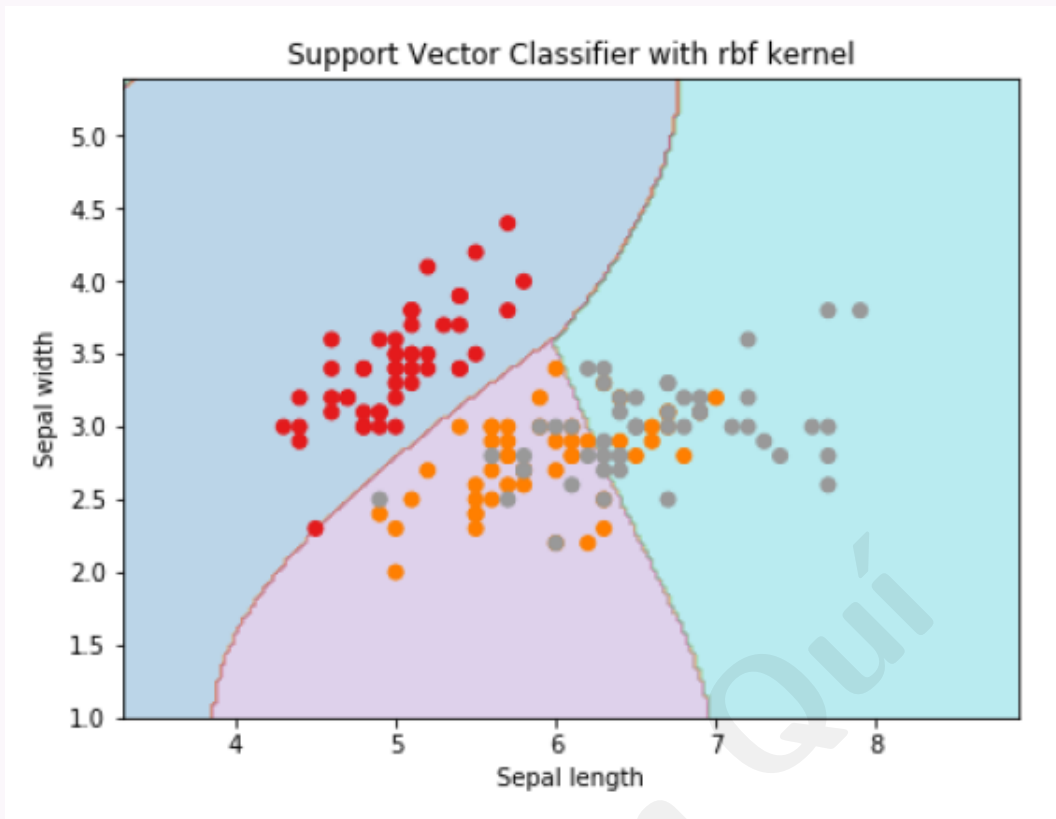


Kernel RBF:

```

svc = svm.SVC(kernel='rbf', gamma='auto', C=C).fit(X, y)

```



6.4 Tinh chỉnh tham số SVM

Tham số quan trọng

kernel: linear, polynomial, RBF, sigmoid. **C:** cân bằng margin vs lỗi phân loại; C lớn $\Rightarrow$ ít sai số hơn, margin có thể nhỏ hơn. Tham số riêng từng kernel: bậc đa thức, γ RBF, v.v. Dùng **cross-validation** (GridSearchCV) để chọn bộ tham số tốt.

GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear', 'poly', 'rbf', 'sigmoid'],
    'degree': [2, 3, 4],
    'gamma': ['scale', 'auto']
}
grid_search = GridSearchCV(SVC(), param_grid, cv=5)
grid_search.fit(X_train, y_train)
print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.best_score_)
```

Output mẫu:

Best parameters: {'C': 0.1, 'degree': 3, 'gamma': 'scale', 'kernel': 'poly'}

Best accuracy: 0.975

6.5 Ưu và nhược điểm

Ưu điểm

Độ chính xác cao; hiệu quả không gian chiều cao; dùng subset điểm huấn luyện nên tiết kiệm bộ nhớ.

Nhược điểm

Thời gian huấn luyện lớn — không phù hợp dataset rất lớn; kém khi các lớp chồng lấn nhiều.

7 Thuật toán Random Forest

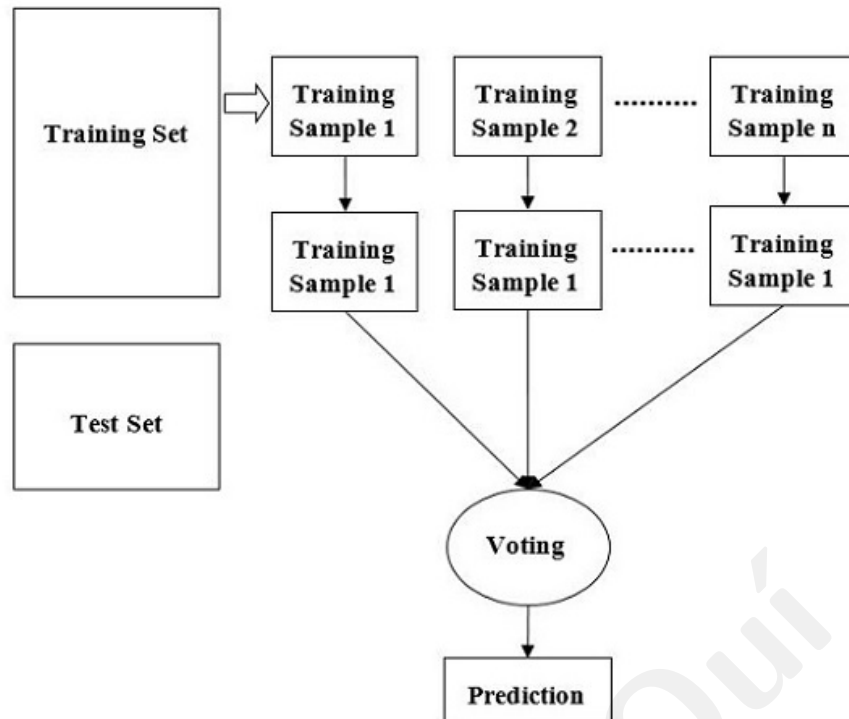
Random Forest

Thuật toán học máy dùng **tập hợp** (ensemble) cây quyết định để dự đoán; Leo Breiman, 2001. Tạo nhiều cây, mỗi cây huấn luyện trên tập con dữ liệu khác nhau; kết hợp dự đoán.

7.1 Cách Random Forest hoạt động

Các bước

1. Chọn ngẫu nhiên mẫu từ dataset.
2. Xây cây quyết định cho từng mẫu; thu dự đoán từng cây.
3. **Bỏ phiếu** (voting) cho từng kết quả dự đoán.
4. Chọn kết quả được bỏ phiếu nhiều nhất làm dự đoán cuối.



Phân loại và hồi quy

Phân loại: dùng **mode** (lớp đa số) của các cây. Hồi quy: dùng **trung bình** dự đoán các cây.

7.2 Ưu điểm

Ưu điểm chính

- **Chống overfitting**: ensemble giảm ảnh hưởng outlier và nhiễu.
- **Độ chính xác cao**: kết hợp nhiều cây.
- **Xử lý dữ liệu thiếu**: không bắt buộc mọi feature có mặt ở mọi điểm.
- **Quan hệ phi tuyến**: cây quyết định mô hình hóa phi tuyến.
- **Feature importance**: hỗ trợ chọn/làm feature.

7.3 Triển khai Python

Bước 1–2: Import và nạp Iris

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
```



```
y = iris.target
```

Bước 3: Tiền xử lý và chia dữ liệu

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.35, random_state=42)
```

Bước 4–5: Huấn luyện và dự đoán

```
rfc = RandomForestClassifier(n_estimators=100)
rfc.fit(X_train, y_train)
y_pred = rfc.predict(X_test)
```

Bước 6: Đánh giá

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall:", recall_score(y_test, y_pred, average='weighted'))
print("F1-score:", f1_score(y_test, y_pred, average='weighted'))
```

Output mẫu:

```
Accuracy: 0.9811320754716981
Precision: 0.9821802935010483
Recall: 0.9811320754716981
F1-score: 0.9811157396063056
```

7.4 Ưu và nhược điểm

Pros

Giảm overfitting nhờ trung bình/kết hợp cây; hoạt động tốt trên nhiều loại dữ liệu hơn một cây; phương sai thấp hơn một cây; linh hoạt, chính xác cao; **không bắt buộc scale** feature.

Cons

Phức tạp cao; xây dựng tốn thời gian và tài nguyên hơn một cây; khó trực quan khi có rất nhiều cây; dự đoán chậm hơn một số thuật toán khác.

8 Confusion Matrix (Ma trận nhầm lẫn)

Confusion Matrix

Cách **dễ nhất** đo hiệu năng bài toán phân loại khi đầu ra có hai hoặc nhiều lớp. Bảng hai chiều “Actual” và “Predicted” với TP, TN, FP, FN.

		Actual	
		1	0
Predicted	1	True Positives (TP)	False Positives (FP)
	0	False Negatives (FN)	True Negatives (TN)

Ví dụ spam email

Spam = **positive** (1); email hợp lệ = **negative** (0).

True Positive (TP)

Thực tế và dự đoán đều 1; mô hình đúng lớp positive (ví dụ email spam đúng là spam).

True Negative (TN)

Thực tế và dự đoán đều 0; đúng lớp negative.

False Positive (FP)

Thực tế 0, dự đoán 1 — **Type I error** (ví dụ email hợp lệ bị gán spam).

False Negative (FN)

Thực tế 1, dự đoán 0 — **Type II error** (spam bị gán hợp lệ).

Phân loại đúng / sai

Đúng: TP và TN. Sai: FP và FN. Từ confusion matrix tính accuracy, precision, recall, v.v.

8.1 Ví dụ thực hành

10 email — nhãn

Positive (spam) = 1; negative (not spam) = 0.

Actual: 0 1 0 1 1 0 0 1 1 1

Predicted: 0 1 0 1 0 1 0 0 1 1

Kết quả: TN TP TN TP FN FP TN FN TP TP

Đếm

TP = 4; FN = 2; FP = 1; TN = 3.

	Actual +	Actual -
Pred +	4 (TP)	1 (FP)
Pred -	2 (FN)	3 (TN)

Trong 10 email: 7 đúng (TP+TN), 3 sai (FP+FN).

8.2 Metric từ confusion matrix

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{7}{10} = 0.7$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{4}{5} = 0.8$$

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{4}{6} \approx 0.667$$

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{3}{4} = 0.75$$

F1 Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \approx 0.727$$

Type I Error Rate

$$\frac{FP}{FP + TN} = \frac{1}{4} = 0.25$$

Type II Error Rate

$$\frac{FN}{FN + TP} = \frac{2}{6} \approx 0.333$$

8.3 Triển khai Python**Thứ tự ma trận sklearn**

`confusion_matrix(y_actual, y_pred)` trả về:

	Pred 0	Pred 1
Actual 0	TN	FP
Actual 1	FN	TP

confusion_matrix

```
from sklearn.metrics import confusion_matrix

y_actual = [0, 1, 0, 1, 1, 0, 0, 1, 1, 1]
y_pred = [0, 1, 0, 1, 0, 1, 0, 0, 1, 1]
cm = confusion_matrix(y_actual, y_pred)
print(cm)
```

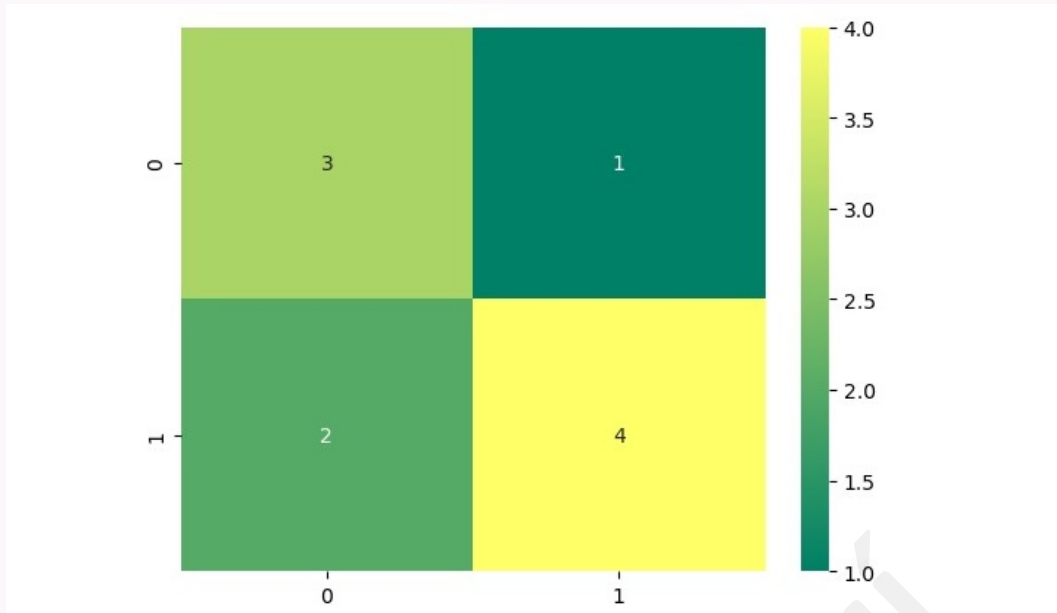
Output:

```
[[3 1]
 [2 4]]
```

Khớp bảng ví dụ trên (TN=3, FP=1, FN=2, TP=4).

Heatmap với seaborn

```
import seaborn as sns
sns.heatmap(cm, annot=True, cmap='summer')
```



Trực x : dự đoán; trực y : thực tế; màu thể hiện số mẫu mỗi ô.

9 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD)

Kỹ thuật tối ưu phổ biến trong học máy: cập nhật tham số (trọng số, bias) lặp lại bằng **từng mẫu huấn luyện** (hoặc mini-batch) thay vì toàn bộ dataset. Biến thể của gradient descent; hiệu quả và nhanh hơn với dataset lớn và thưa.

9.1 Gradient Descent

Gradient Descent

Thuật toán tối ưu tối thiểu hóa **hàm cost** của mô hình: tính gradient cost theo tham số, cập nhật tham số ngược hướng gradient.

9.2 SGD

SGD

Biến thể cập nhật tham số sau **mỗi ví dụ** (hoặc mini-batch nhỏ), không đợi duyệt hết dataset mỗi bước. Hội tụ nhanh hơn và cần ít bộ nhớ lưu toàn bộ dữ liệu hơn batch gradient descent.

Thuật toán SGD

1. Khởi tạo tham số ngẫu nhiên.
2. Mỗi epoch: xáo trộn dữ liệu huấn luyện.
3. Với mỗi mẫu: tính gradient cost theo tham số; cập nhật tham số ngược gradient.
4. Lặp đến hội tụ.

Cập nhật trọng số

$$w := w - \alpha \nabla_w J(w; x_i, y_i)$$

x_i, y_i : mẫu và nhãn; α : learning rate; J : hàm loss; $\nabla_w J$: gradient của J theo w .

Khác batch gradient descent

SGD dùng **một** mẫu (hoặc mini-batch) để tính gradient; batch GD dùng **toàn bộ** dataset mỗi bước.

9.3 Ví dụ Python — SGDClassifier trên Iris**Triển khai đầy đủ**

Dự đoán loài hoa Iris từ hai feature sepal width và sepal length:

```
import numpy as np
from sklearn import datasets
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import metrics

iris = datasets.load_iris()
X, y = iris.data[:, :2], iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=1)

scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

clf = SGDClassifier(alpha=0.001, max_iter=200)
clf.fit(X_train, y_train)

y_train_pred = clf.predict(X_train)
print("The Accuracy of SGD classifier is:",
      metrics.accuracy_score(y_train, y_train_pred) * 100)
```

Output mẫu:

The Accuracy of SGD classifier is: 73.33333333333333

9.4 Ứng dụng

Ứng dụng SGD

SGD là kỹ thuật tối ưu, không phải mô hình ML hoàn chỉnh; dùng rộng rãi khi dữ liệu **thừa** — đặc biệt phân loại văn bản, NLP. Mở rộng tốt cho bài toán hàng chục nghìn mẫu và hàng chục nghìn feature.

9.5 Ưu điểm và thách thức

Ưu điểm

- **Hiệu quả bộ nhớ:** xử lý theo batch nhỏ.
- **Hội tụ nhanh** trên dataset lớn so với batch GD.
- **Thoát cực tiểu cục bộ:** tính ngẫu nhiên giúp tìm nghiệm tốt hơn đôi khi.

Thách thức

- **Gradient nhiễu:** có thể làm chậm hội tụ.
- **Chọn learning rate:** quan trọng cho tối ưu ổn định.
- **Kích thước mini-batch:** ảnh hưởng tốc độ hội tụ và ổn định.